



Predicting Short-Term CPU Thermal Events from System Telemetry

Statistical Machine Learning

Chenjie Liu, Jacopo Francesco Calvano, Kayo Yokoyama Reis, Saif Ali
Sapienza University of Rome

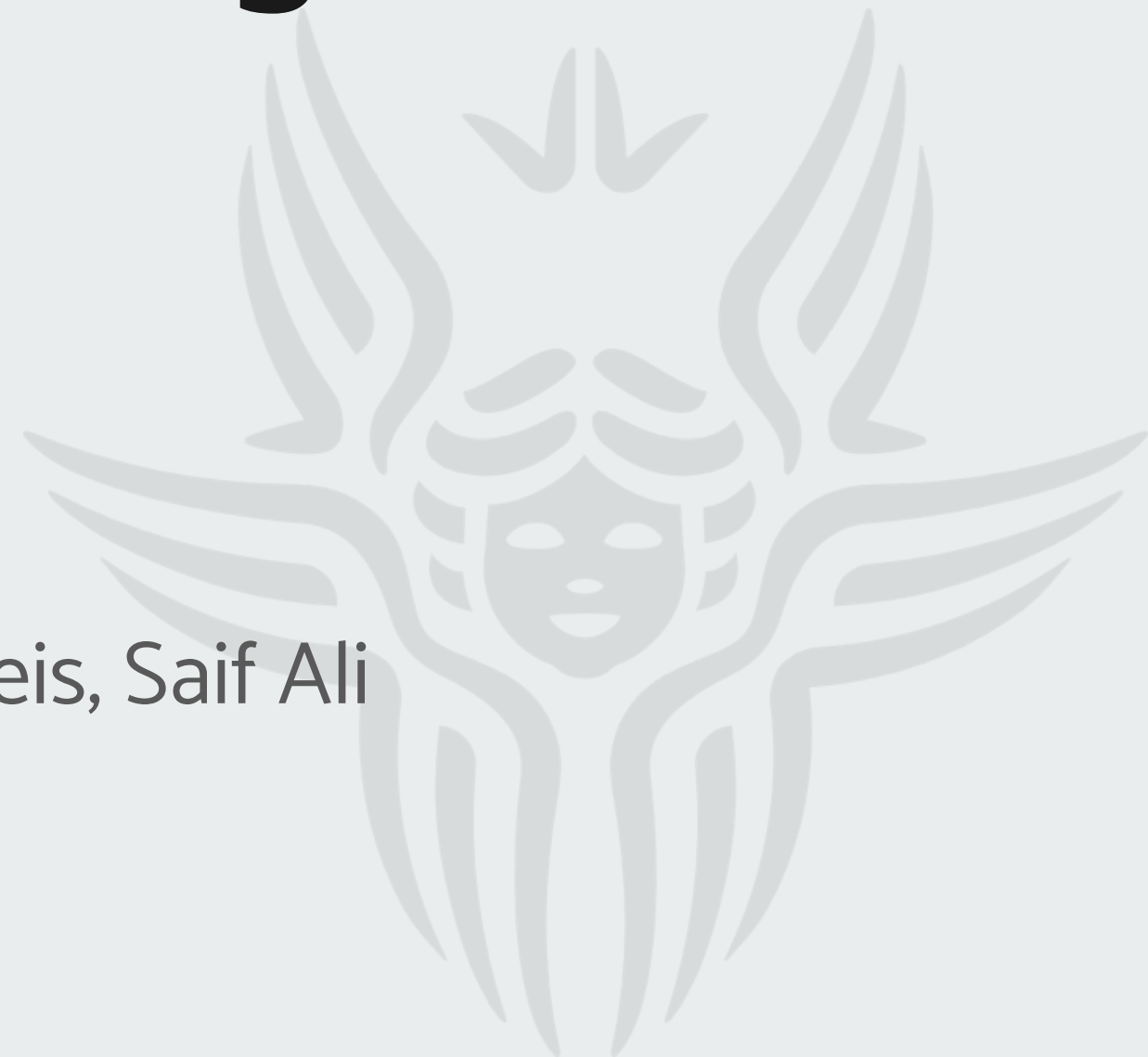




Table of contents

1	Introduction & Problem Definition
3	Data Preprocessing & Feature Engineering
5	Results & Model Comparison
7	Limitations & Future Work

2	Dataset & Data Collection
4	Models and Experimental Setup
6	Discussion
8	Conclusion





Introduction and Dataset

Context

Modern laptops automatically reduce CPU performance when temperatures become too high (thermal throttling). Predicting these thermal events in advance can improve system monitoring and resource management.

Goal

Predict whether the CPU temperature will exceed 75°C within the next two minutes using only system telemetry such as temperatures, power consumption, CPU/GPU activity, clock frequencies and memory usage.





Dataset Overview

Data Collection

- Telemetry collected from 2 Apple Silicon laptops (M1 and M5)(22 sessions, 45,328 labelled samples).
- Measurements recorded every 5 seconds during stress and cool-down sessions.

Recorded Features

- CPU die temperature
- CPU & GPU power consumption
- CPU frequencies
- P-core activity
- Memory usage and system load

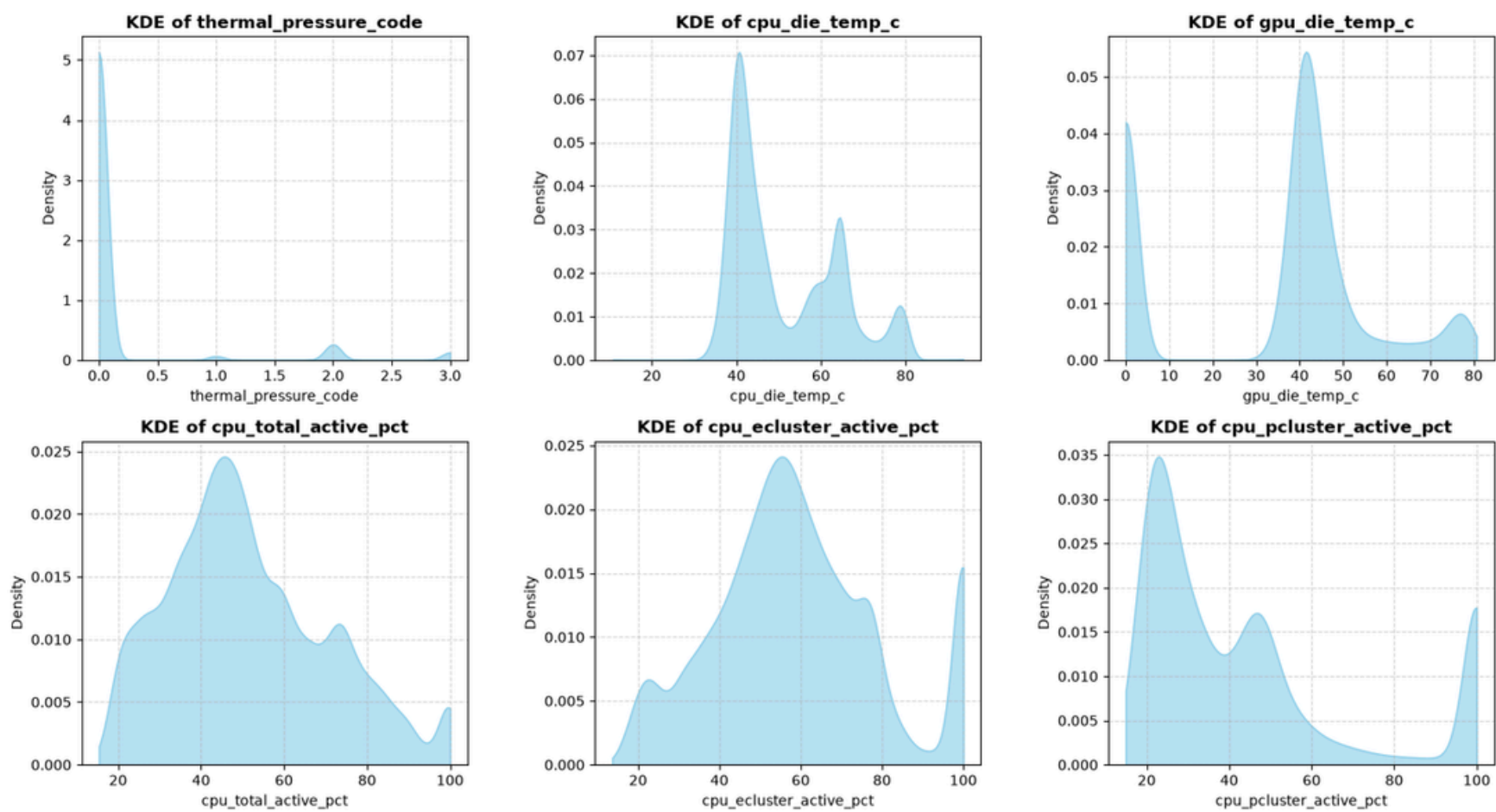
Prediction Task

Predict whether the CPU temperature will exceed 75°C within the next 2 minutes using the current system telemetry.

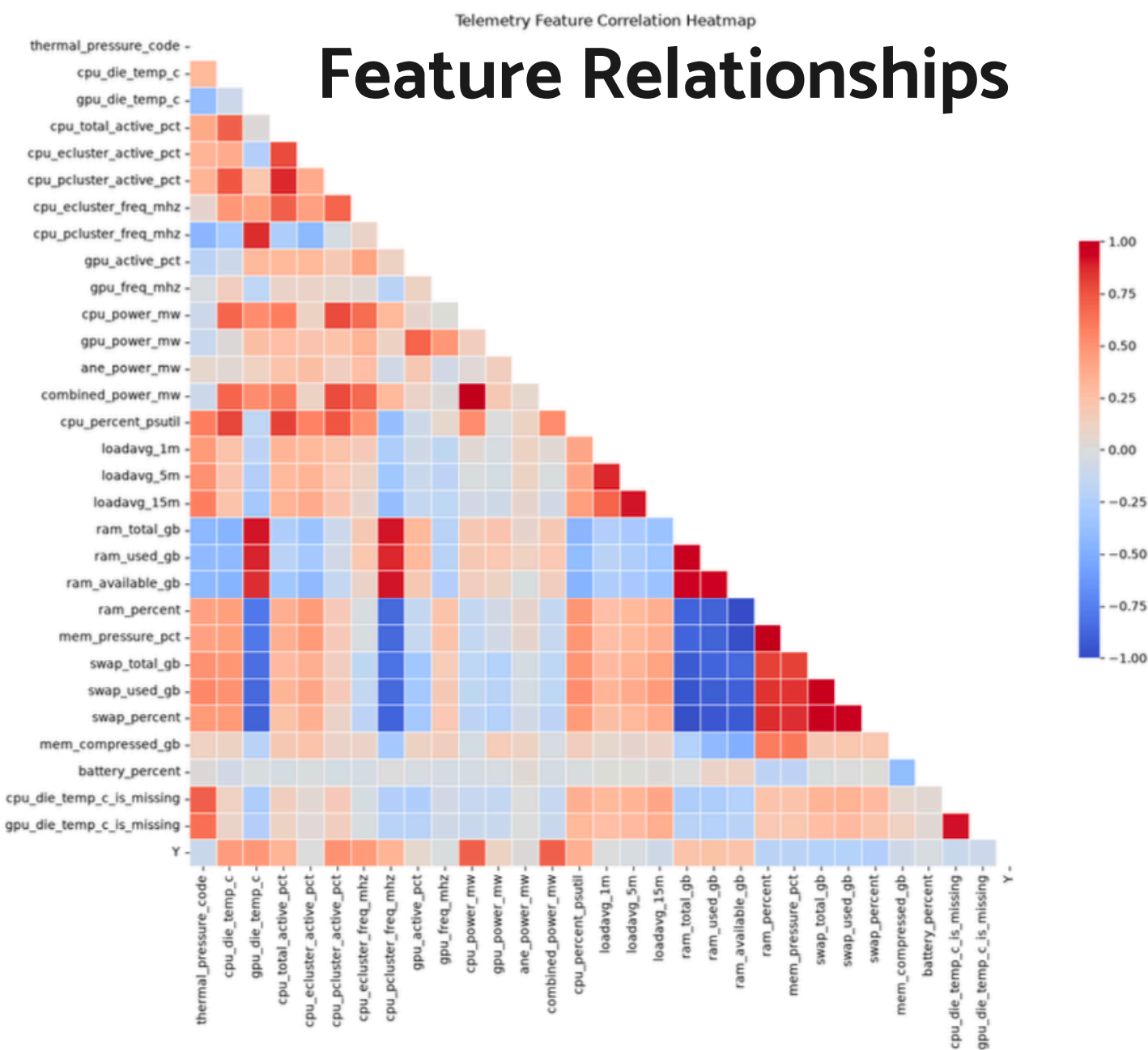


Data Exploration (EDA)

Feature Distribution



Feature Relationships



Data Preprocessing & Feature Engineering

Data Cleaning

The raw telemetry was first cleaned by removing incomplete samples at the beginning and end of each session. Missing values were forward-filled when possible, while missing CPU temperatures were imputed and marked with an additional flag.

Label Generation

The prediction target was defined as a binary variable: a sample is labelled as 1 if the CPU reaches 75°C within the next two minutes, while samples without enough future observations were discarded.

Feature Engineering

To capture the temporal behaviour of the system, we generated lag features, rolling statistics, rates of change and interaction features. After feature selection using Random Forest and permutation importance, the final dataset contained 24 features.



Temporal Structure and Prediction Horizon

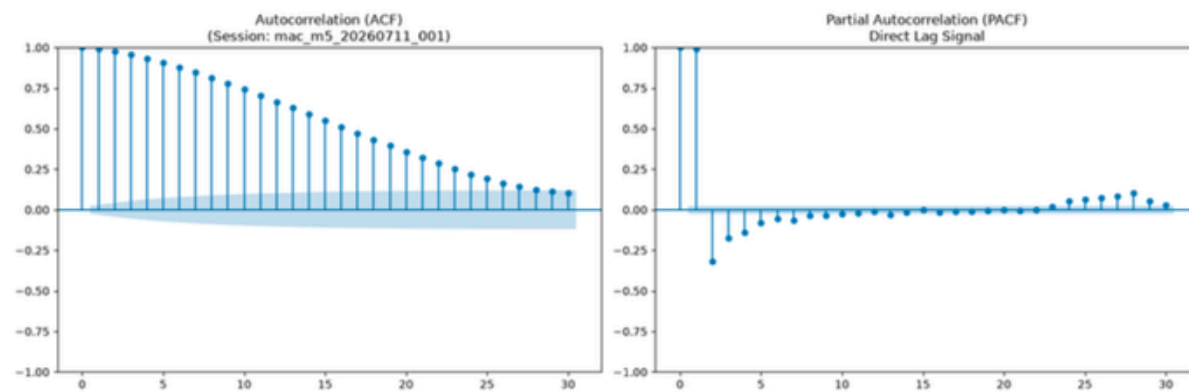
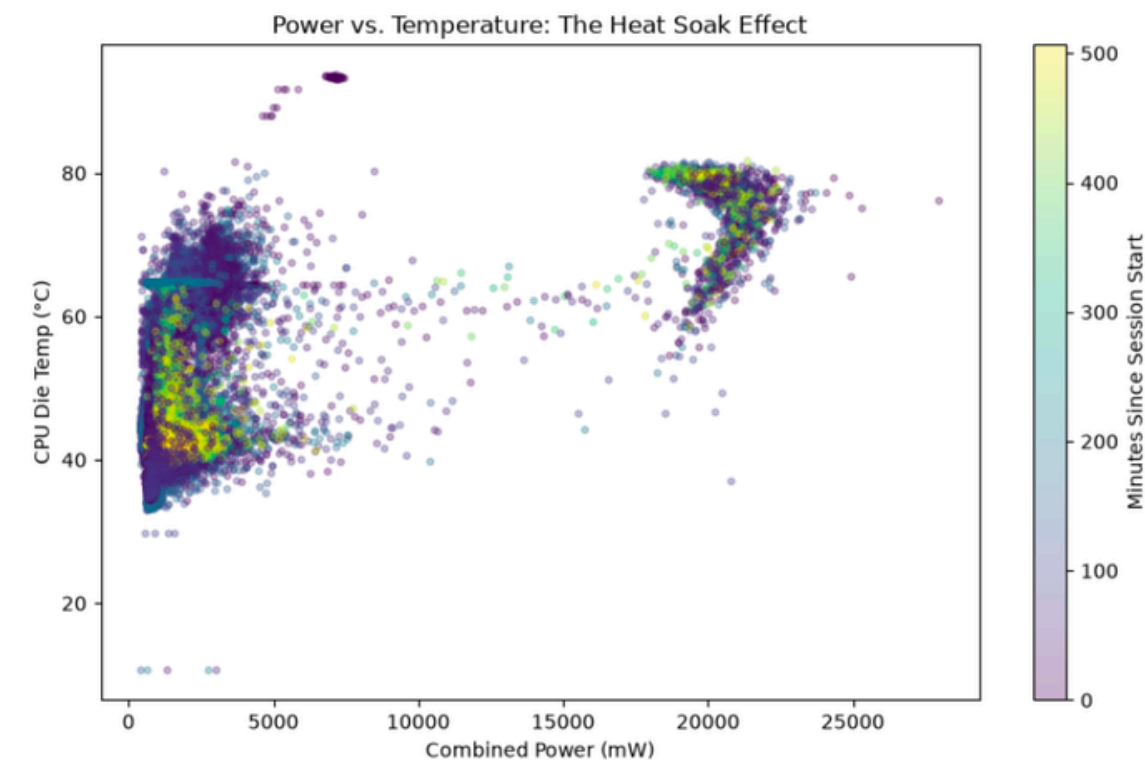


Figure 4: ACF and PACF of die temperature over 30 lags (150 s).

Autocorrelation Analysis

CPU temperature is highly autocorrelated because of thermal inertia. The ACF decays slowly over 30 lags (150 seconds), showing that consecutive observations are strongly dependent



Heat Soak Effect

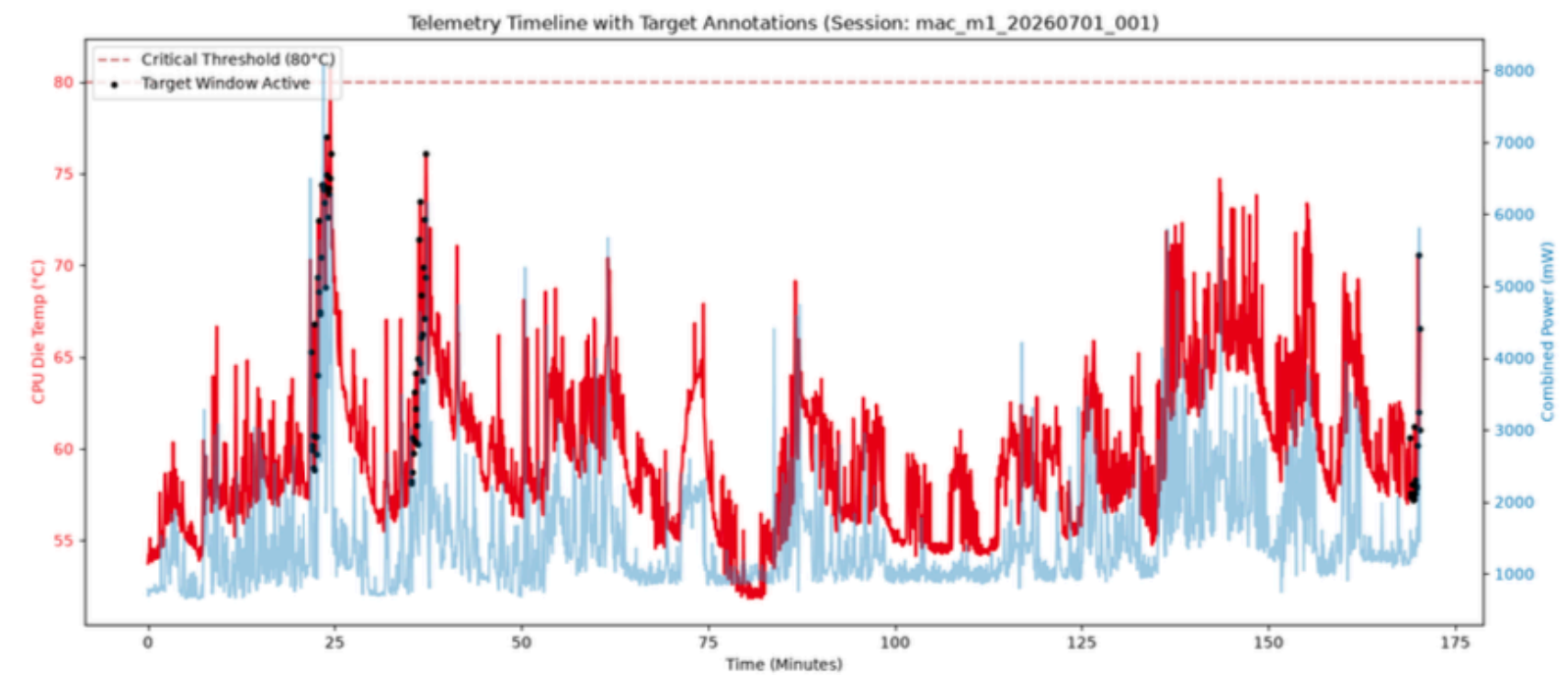
The same power level can produce different temperatures depending on how long the CPU has been under load. This hysteresis motivates the use of rolling-window features instead of relying only on instantaneous measurements.



Feature Engineering

Generated features:

- Lag features
- Rolling mean and standard deviation
- Rolling maximum and minimum
- First differences and accelerations
- Interaction features



Key IDEA

Instead of predicting from a single observation, the model learns from the recent thermal history of the CPU. Temporal features allow it to anticipate overheating events before they occur.



Feature Selection

Why Feature Selection?

Not all engineered features contribute equally to predicting thermal events. We evaluated feature importance to reduce redundancy and keep only the most informative variables.

Selection Process

- Random Forest impurity importance
- Permutation importance on the validation set
- Importance aggregated by feature family
- Final model built using 24 selected features

	total_importance	mean_importance	n_features
cpu_die_temp_c	0.039365	0.001458	27
other	0.012597	0.000525	24
cpu_pcluster_active_pct	0.003059	0.000139	22
ram_used_gb	0.001414	0.001414	1
ram_total_gb	0.000000	0.000000	1
cpu_pcluster_freq_mhz	-0.006837	-0.000263	26
cpu_power_mw	-0.013060	-0.000502	26
combined_power_mw	-0.032019	-0.001455	22

Key Findings

- CPU die temperature was the most informative feature family, providing the strongest predictive signal.
- Power features scored negative permutation importance, yet CPU power is the variable most correlated with the target ($r = 0.71$)
- Based on the importance analysis, the model retained the most informative features for training.



Experimental Design

Split	Sessions	Samples	Positives	Rate
Train	15	30,809	4,402	14.3%
Validation	3	8,524	1,465	17.2%
Test	4	5,357	1,027	19.2%

”We decided to split the data by session instead of by individual samples because consecutive measurements are highly correlated. If we randomly split the rows, the model could see almost identical observations during training and testing, leading to overly optimistic results.”



Machine Learning Models

Model Selection

To evaluate different learning strategies, we compared four machine learning models with different levels of complexity:

- **Logistic Regression** – linear baseline model.
- **Random Forest** – ensemble of decision trees using bagging.
- **Gradient Boosting** – sequential ensemble optimized to reduce prediction errors.
- **Neural Network** – small feed-forward architecture implemented in PyTorch.

Logistic Regression – Baseline
Logistic Regression – Class Weights
Logistic Regression – Random Oversampling
Logistic Regression – SMOTE
Logistic Regression – Thresh Moving ($t=0.55$)
Random Forest – Baseline
Random Forest – Class Weights
Random Forest – Random Oversampling
Random Forest – SMOTE
Random Forest – Thresh Moving ($t=0.69$)
Gradient Boosting – Baseline
Gradient Boosting – Class Weights
Gradient Boosting – Random Oversampling
Gradient Boosting – SMOTE
Gradient Boosting – Thresh Moving ($t=0.55$)
Neural Network – Baseline
Neural Network – Class Weights
Neural Network – Random Oversampling
Neural Network – SMOTE
Neural Network – Thresh Moving ($t=0.60$)



Handling Class Imbalance

Technique	LR	RF	GB	NN
Baseline	0.787	0.782	0.789	0.789
Class weights	0.652	0.798	0.704	0.674
Random oversamp.	0.650	0.793	0.703	0.739
SMOTE	0.658	0.787	0.707	0.728
Threshold moving	0.788	0.787	0.791	0.792

Thermal events represent only a small fraction of the dataset, making the classification task highly imbalanced. To address this issue, we tested different strategies, including **class weighting, random oversampling, SMOTE, and threshold moving**.

Although some methods improved recall, they generally reduced precision and did not lead to a significant increase in the overall F1-score. As a result, we selected the original training strategy without resampling, since it provided the best balance between precision and recall.





Model Performance

	Predicted 0	Predicted 1
Actual 0	4,315	15
Actual 1	404	623

The confusion matrix provides a more detailed interpretation of the model's behaviour. Out of 1,027 positive events, the model correctly detected 623, while missing 404. At the same time, it generated only 15 false positives, confirming that the classifier is highly conservative and prioritises precision over recall.

	Persistence	Random forest
Precision	0.948	0.976
Recall	0.411	0.607
F_1	0.573	0.750
ROC-AUC	n/a	0.910
PR-AUC	n/a	0.835

Results

The Random Forest achieved the best overall performance on the test set, outperforming the persistence baseline across all evaluation metrics. In particular, the model reached an F1-score of 0.750, compared with 0.573 for the baseline, while maintaining a very high precision (0.976) and a ROC-AUC of 0.910. **These results indicate that the model can reliably identify future thermal events while producing very few false alarms.**



Main Failures & Course Corrections

Main Failures

- **Cross-platform telemetry:** We planned to use macOS and Linux. But, the semantics differed, so Apple-specific columns were entirely missing on Linux.
- **Prediction horizon:** A 1-minute horizon made the task too naive. The model mainly extrapolated the current temperature instead of anticipating future events.
- **Class imbalance:** Thermal events were much less frequent than normal samples. Several balancing techniques improved recall but reduced precision, leading to little or no overall gain.
- **Misleading labels:** We first gave the model the current CPU temperature to predict future temperature. Since temperature changes slowly, the model could "cheat" by leaning on the present value instead of learning to anticipate.

Course Corrections

- Dropped Linux and scoped the project to macOS, since that is the data we could grow.
- Increased the prediction horizon from 1 minute to 2 minutes.
- Evaluated different imbalance handling strategies and retained the original dataset when resampling failed to improve performance.
- Removed the current temperature from the inputs, keeping only past temperature trends.





Discussion

The experimental results show that telemetry data can effectively predict short-term CPU thermal events, achieving high precision while maintaining an average recall. However, the model still misses a significant number of positive events, suggesting that detecting the earliest stages of CPU overheating remains a challenging task.

Another important finding is that all four machine learning models achieved very similar validation performance. This indicates that the current limitation is not the choice of the algorithm, but rather the quality of the available data, the label definition, and the evaluation strategy. Future improvements are therefore more likely to come from better data collection and feature design than from adopting more complex models.



Limitations & Future Work

Current Limitations

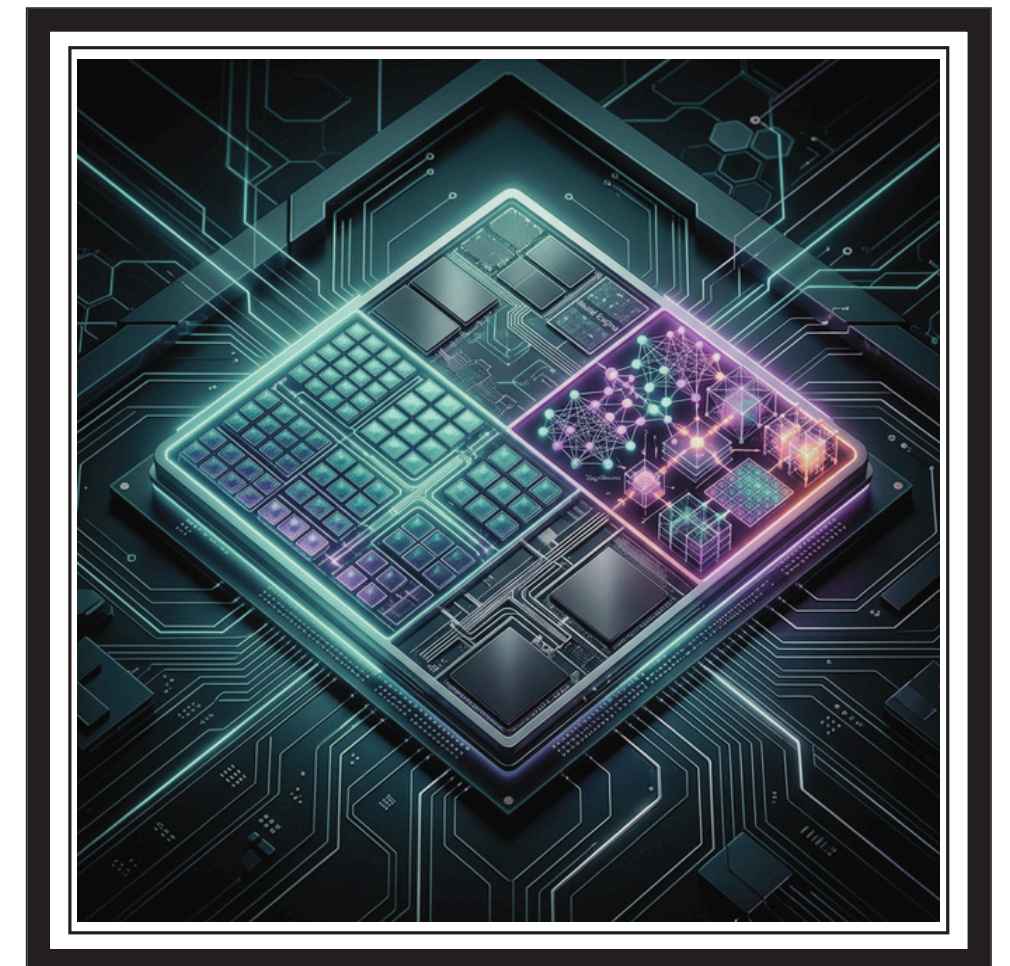
Despite the encouraging results, several limitations remain:

- Imputed labels: around 10.9% of CPU temperature values were reconstructed before label generation.
- Limited generalization: the dataset includes only two Apple Silicon devices.
- Session variability: only 22 sessions were available, limiting the evaluation across different workloads.
- Row-level evaluation: performance was measured on individual samples rather than complete thermal events.

Future Work

To improve the system, future work will focus on:

- Event-level evaluation instead of row-level metrics.
- Optimizing the classification threshold.
- Session-wise cross-validation.
- Per-device evaluation.
- Collecting additional data and extending compatibility to Linux-based systems.





Thank you!

